



Descripción General

En esta tarea deberás implementar un programa que simule una dinámica de ataques entre piezas reina en un tablero de ajedrez de 8x8. Las reinas se turnan para atacar a otras, y el juego termina cuando ninguna puede atacar más. El programa también permitirá ejecutar y revisar múltiples juegos, almacenando sus resultados para navegación posterior.

Reglas del juego

1. Tablero y posiciones:

- El tablero tiene dimensiones fijas de 8 columnas por 8 filas.
- Las posiciones se expresan en notación estándar de ajedrez: una letra entre **a** y **h** (columna) seguida de un número entre 1 y 8 (fila). Ejemplo: **e4**, **a1**, **h8**.

2. Ingreso de datos:

- El programa recibe como entrada un único string con las posiciones iniciales de las reinas, separadas por espacio. Ejemplo: **d1 h5 b7 f6**
- Todas las posiciones deben ser válidas y distintas.

3. Turnos:

- Las reinas están numeradas desde la 1 hasta la N según el orden en que fueron ingresadas.
- En cada turno, la reina con el siguiente número juega, en orden creciente. Si se llega a la reina N, el orden continúa desde la 1.
- Si una reina ha sido eliminada, se omite en la secuencia.
- Antes de iniciar los turnos, el programa debe mostrar el estado inicial del tablero en formato texto.
- En cada turno se debe indicar qué reina juega. Si ataca, se indica a qué reina elimina y a qué casilla se mueve.
- Si ocurre un ataque y movimiento, se debe imprimir el tablero actualizado inmediatamente después del turno.
- Si la reina no puede atacar a nadie, se indica que pasa su turno.

4. Ataques:

- En su turno, la reina activa revisa a las demás reinas en orden creciente de numeración (circular), buscando una que esté a su alcance para atacar.
- Una reina puede atacar si la otra está en la misma fila, columna o diagonal, sin que haya otras reinas entre medio.
- Cada reina puede realizar a lo más un ataque por turno: si encuentra una reina alcanzable, la elimina y se mueve a ocupar su posición.

- Si no puede atacar a nadie, simplemente pasa su turno.

5. Término del juego:

- El juego termina cuando se completa un ciclo completo de turnos sin que ninguna reina logre atacar.
- Al finalizar, el programa debe mostrar:
 - El tablero final en formato texto, con la ubicación de las reinas sobrevivientes.
 - La lista de reinas sobrevivientes con su número y posición final.
- Las casillas vacías pueden representarse con un punto (.), y las ocupadas con el número de la reina correspondiente.

Funcionalidad de múltiples juegos y menú

- Al ejecutar el programa, el usuario debe interactuar con un menú de texto que permita:
 1. Iniciar un nuevo juego.
 2. Revisar un juego anterior.
 3. Salir del programa.
- Cada juego debe almacenarse en un diccionario, donde:
 - La clave es una cadena ingresada por el usuario que identifica el juego.
 - El valor es una lista de strings que contiene los mensajes impresos durante la ejecución del juego.
- Al revisar un juego anterior, el usuario puede ingresar el nombre y ver toda la secuencia guardada (movimientos y tableros).

Puedes ver en el anexo de este enunciado tres ejemplos que te servirán para revisar el funcionamiento de tu propio programa.

Código base entregado, código requerido y puntajes

Se entrega un código base (`tm3v1-base.py`) que contiene los encabezados (*stubs*), e implementaciones parciales – y en algunos casos completas – de las funciones del programa. Tendrás libertad de definir funciones auxiliares adicionales. Las funciones mínimas requeridas son:

- `menu_principal()`: gestiona el menú de texto principal del programa. Se da completa en el código base.
- `jugar_partida(nombre_juego, historial_juegos)`: ejecuta una nueva partida y almacena su historial. Se da completa en el código base.
- `[0,5] leer_posiciones_entrada()`: Solicita al usuario que ingrese un string con las posiciones iniciales de las reinas, separadas por espacio, utilizando notación estándar de ajedrez (como `a1`, `d5`, `h8`). La función debe procesar esta entrada y validarla antes de retornarla.

La validación incluye:

- Verificar que se haya ingresado al menos una posición.

- Comprobar que cada posición tenga exactamente dos caracteres: el primero una letra entre **a** y **h** (columna), y el segundo un dígito entre **1** y **8** (fila).
- Asegurar que todas las posiciones sean distintas (sin duplicados).

Si alguna validación falla, se debe mostrar un mensaje de error y volver a solicitar la entrada. Una vez validada, la función retorna una lista de strings con las posiciones ingresadas en el mismo orden, que luego serán numeradas como Reina 1, Reina 2, etc.

Ejemplo:

Si el usuario ingresa:

Ingresar las posiciones iniciales de las reinas (separadas por espacio): d1 h5 b7 f6

La función retornará la lista:

```
["d1", "h5", "b7", "f6"]
```

Si en cambio el usuario ingresa una posición fuera del tablero (como **h9**) o repite una posición (como **b7 b7**), la función mostrará un mensaje de error indicando las posiciones inválidas y repetidas, y pedirá ingresar nuevamente los datos.

- `[0,5] convertir_a_coordenadas(pos)`: convierte notación tipo **e4** a coordenadas internas. Se debe completar la implementación dada en el código base.
- `[1,0] es_ataque_valido(origen, destino, posiciones)`: Determina si una reina ubicada en la posición **origen** puede atacar a una reina ubicada en la posición **destino**, considerando el estado actual del tablero. La función recibe las posiciones en notación estándar de ajedrez (como **d4**, **g6**, etc.), y un diccionario **posiciones** que asocia a cada reina activa (identificada por número) con su posición actual.

La función debe verificar si:

- el movimiento entre origen y destino está alineado en la misma fila, columna o diagonal,
- y no existe ninguna otra reina entre medio que obstruya el camino (es decir, el camino está libre).

Devuelve **True** si el ataque es posible (línea recta sin obstrucciones); en caso contrario, devuelve **False**.

- `[1,0] mostrar_tablero(posiciones)`: Recibe un diccionario **posiciones** que asocia a cada reina (identificada por su número) con su posición actual en notación estándar de ajedrez (por ejemplo, **d4**). La función debe construir una representación textual del tablero de 8×8, donde cada casilla vacía se representa con un punto (.) y cada casilla ocupada muestra el número de la reina que la ocupa.

El tablero se imprime y también se devuelve como un string multilínea, con una línea por fila del tablero, en orden desde la fila superior (8) hasta la inferior (1), y de izquierda a derecha desde la columna **a** hasta la **h**. El formato debe permitir visualizar claramente las posiciones ocupadas por las reinas y distinguir las casillas vacías.

Ejemplo:

Si el diccionario recibido es:

```
{1: "a1", 2: "h8", 3: "d4"}
```

El tablero producido será similar a:

```
. . . . . 2
. . . . .
. . . . .
. . . 3 . . .
. . . . .
. . . . .
. . . . .
1 . . . . .
```

Cada número representa a una reina identificada por su número. Las filas aparecen de arriba hacia abajo (de 8 a 1), y las columnas de izquierda a derecha (de a a h).

- [3,0] `simular_turnos(posiciones_iniciales, historial)`: Simula el desarrollo completo de una partida a partir de la lista de posiciones iniciales, que debe convertirse internamente en un diccionario que asocie a cada reina (numerada desde 1) con su posición. En cada turno, la reina activa recorre el conjunto de reinas vivas en orden creciente (circularmente) para intentar atacar a la primera que esté a su alcance según las reglas del ajedrez (movimiento en línea recta sin obstrucciones, en fila, columna o diagonal). Si logra atacar, se mueve a la posición de la reina eliminada; si no, pasa su turno. El juego termina cuando se completa un ciclo completo sin que ninguna reina logre atacar.

Cada evento importante debe agregarse a la lista `historial` como un string, incluyendo¹:

- el número y posición de la reina que juega en cada turno,
 - la descripción del ataque realizado (si corresponde) o la indicación de que no pudo atacar,
 - el tablero en formato texto tras cada movimiento exitoso,
 - el tablero final y la lista de reinas sobrevivientes al término del juego.
- `revisar_partida(historial_juegos)`: permite consultar partidas anteriores guardadas. Se da completa en el código base.

Sistema de Evaluación

Cada ítem es evaluado con una escala de 1-5 (1: no presenta solución, 2: presenta esbozo de solución incompleto o con errores graves, 3: solución incompleta o imprecisa, 4: error(es) mínimo(s), 5: excelente) y luego un ponderador (1:0, 2:0,25, 3:0,5, 4:0,75, 5:1,0) es aplicado al puntaje del ítem. El puntaje otorgado a la solución se calcula como la suma de los puntajes parciales obtenidos, aplicándose los ponderadores. La nota final es el puntaje calculado más el punto base.

Consideraciones importantes para ejecución y entrega

El código base de la tarea se distribuye con nombre `tm3v1-base.py`. Para desarrollar tu tarea, puedes cambiarle el nombre a `tm3v1.py`. Debes subir al buzón de entrega en Canvas un archivo ZIP cuyo nombre debe ser tu RUT sin puntos, ni guión, ni dígito verificador, p.ej., `22543161.zip`. Dicho archivo ZIP debe contener el archivo de código de tu tarea `tm3v1.py`.

¹Ver salida de la simulación en los anexos al final de este enunciado para mayor claridad sobre el resultado esperado

Integridad Académica

Citamos a continuación las disposiciones de ética académica descritas en el *syllabus* del curso:

En las evaluaciones del curso, se espera que los estudiantes presenten código original de su propia autoría. El plagio, entendido como la presentación de trabajo ajeno como propio, constituye una falta grave a la ética académica. Esto incluye, pero no se limita a: la compra de tareas, la comunicación con terceros por cualquier medio durante las evaluaciones para obtener soluciones totales o parciales, la publicación o copia de código de evaluaciones en foros, el uso de modelos de lenguaje para generar el código requerido, entre otros.

Las sospechas de falta a la integridad académica en evaluaciones de laboratorio o tareas serán investigadas por los profesores del curso. En caso de confirmarse la falta, la sanción consistirá en la anulación del puntaje obtenido o, en el caso de una tarea, la asignación de una calificación de 1,0. Los estudiantes podrán apelar esta decisión; sin embargo, al hacerlo, el caso será escalado a la Fiscalía Universitaria para la apertura de un proceso investigativo. Durante este proceso, el estudiante tendrá derecho a defensa, y el Juez de la Facultad de Ingeniería y Ciencias Aplicadas determinará la sanción correspondiente en caso de comprobarse la infracción. Sin embargo, cabe destacar que la apelación y el correspondiente escalamiento a la Fiscalía pueden retrasar en aproximadamente 30 días el resultado final (la nota final) del alumno(a) en el curso, lo cual podrá afectar el proceso de “toma de ramos” del alumno(a) para el semestre siguiente.

Las faltas de mayor gravedad, como la copia en un examen o la reincidencia en evaluaciones de laboratorio o tareas, serán reportadas directamente a la Fiscalía Universitaria, que llevará adelante el proceso antes mencionado. En estos casos, las sanciones pueden variar según la gravedad de la infracción, incluyendo desde la asignación de una nota 1,0 en la evaluación hasta la expulsión de la carrera. Dicho proceso en Fiscalía Universitaria retrasará el resultado final del alumno(a) en el curso.

Además, téngase presente lo siguiente:

Es importante recalcar que, en las evaluaciones, se espera que los estudiantes apliquen únicamente los conocimientos y habilidades adquiridos hasta la clase inmediatamente anterior a la evaluación. No se permite el uso de técnicas o conocimientos que excedan el alcance de lo enseñado hasta ese punto, incluyendo pero no limitándose a enfoques de programación avanzados, paradigmas no cubiertos en el curso (como la programación orientada a objetos o la programación funcional), o el uso de lenguajes de programación distintos a los que se han enseñado.

Anexos

1. Simulación para posiciones iniciales: a1 h1 a8 h8

```
=== Menú Principal ===
1. Iniciar nuevo juego
2. Revisar juego anterior
3. Salir
Elige una opción (1-3): 1
Nombre del juego: cuadrado
Ingresa las posiciones iniciales de las reinas (separadas por espacio): a1 h1 a8 h8
3 . . . . . 4
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
```

```

1 . . . . . 2

3 . . . . . 4
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . . 1

. . . . . 3
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . . 1

. . . . . 1
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .

. . . . . 1
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .

=== Menú Principal ===
1. Iniciar nuevo juego
2. Revisar juego anterior
3. Salir
Elige una opción (1-3): 2
Nombre del juego a revisar: cuadrado
=== Revisión de la partida 'cuadrado' ===
=== Estado inicial del tablero ===
3 . . . . . 4
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
1 . . . . . 2
Turno de la Reina 1 (a1)
  Ataca a Reina 2 en h1
3 . . . . . 4
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . . 1
Turno de la Reina 3 (a8)
  Ataca a Reina 4 en h8
. . . . . 3
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .

```

```

. . . . . 1
. . . . . 1
Turno de la Reina 1 (h1)
  Ataca a Reina 3 en h8
. . . . . 1
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
Turno de la Reina 1 (h8)
  No puede atacar a nadie. Pasa su turno.
Fin del juego. Estado final:
. . . . . 1
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
Reinas sobrevivientes:
  Reina 1 en h8

```

2. Simulación para posiciones iniciales: b1 f2 h3 c4 a5 d6 g7 e8

Nota: esta es una solución conocida al “problema de las ocho reinas”, es decir, ocho reinas puestas en el tablero tal que ninguna es capaz de atacar a las demás. El juego termina inmediatamente después que se completa la primera ronda de ataques, pues ninguna reina es capaz de atacar a las demás.

```

=== Menú Principal ===
1. Iniciar nuevo juego
2. Revisar juego anterior
3. Salir
Elige una opción (1-3): 1
Nombre del juego: ocho reinas
Ingresa las posiciones iniciales de las reinas (separadas por espacio): b1 f2 h3 c4 a5 d6 g7 e8
. . . . 8 . . .
. . . . . 7 .
. . . 6 . . . .
5 . . . . .
. . 4 . . . .
. . . . . 3
. . . . . 2 .
. 1 . . . . .

. . . . 8 . . .
. . . . . 7 .
. . . 6 . . . .
5 . . . . .
. . 4 . . . .
. . . . . 3
. . . . . 2 .
. 1 . . . . .
=== Menú Principal ===
1. Iniciar nuevo juego
2. Revisar juego anterior
3. Salir
Elige una opción (1-3): 2
Nombre del juego a revisar: ocho reinas
=== Revisión de la partida 'ocho reinas' ===
=== Estado inicial del tablero ===
. . . . 8 . . .
. . . . . 7 .
. . . 6 . . . .
5 . . . . .
. . 4 . . . .

```

```

. . . . . 3
. . . . . 2 . .
. 1 . . . . .
Turno de la Reina 1 (b1)
  No puede atacar a nadie. Pasa su turno.
Turno de la Reina 2 (f2)
  No puede atacar a nadie. Pasa su turno.
Turno de la Reina 3 (h3)
  No puede atacar a nadie. Pasa su turno.
Turno de la Reina 4 (c4)
  No puede atacar a nadie. Pasa su turno.
Turno de la Reina 5 (a5)
  No puede atacar a nadie. Pasa su turno.
Turno de la Reina 6 (d6)
  No puede atacar a nadie. Pasa su turno.
Turno de la Reina 7 (g7)
  No puede atacar a nadie. Pasa su turno.
Turno de la Reina 8 (e8)
  No puede atacar a nadie. Pasa su turno.
Fin del juego. Estado final:
. . . . 8 . . .
. . . . . 7 .
. . . 6 . . . .
5 . . . . . .
. . 4 . . . .
. . . . . 3
. . . . . 2 . .
. 1 . . . . .
Reinas sobrevivientes:
  Reina 1 en b1
  Reina 2 en f2
  Reina 3 en h3
  Reina 4 en c4
  Reina 5 en a5
  Reina 6 en d6
  Reina 7 en g7
  Reina 8 en e8

```

```

=== Menú Principal ===
1. Iniciar nuevo juego
2. Revisar juego anterior
3. Salir
Elige una opción (1-3):

```

3. Simulación para posiciones iniciales: d4 d1 d7 a4 g4 a1 g7

```

=== Menú Principal ===
1. Iniciar nuevo juego
2. Revisar juego anterior
3. Salir
Elige una opción (1-3): 1
Nombre del juego: estrella
Ingresa las posiciones iniciales de las reinas (separadas por espacio): d4 d1 d7 a4 g4 a1 g7
. . . . .
. . . 3 . . 7 .
. . . . .
. . . . .
4 . . 1 . . 5 .
. . . . .
. . . . .
6 . . 2 . . . .

. . . . .
. . . 3 . . 7 .
. . . . .
. . . . .
4 . . . . 5 .
. . . . .
. . . . .
6 . . 1 . . . .

```



```

. . . . .
. . . . . 7 .
. . . . .
. . . . .
3 . . . . . 5 .
. . . . .
. . . . .
6 . . 1 . . . .

. . . . .
. . . . . 5 .
. . . . .
. . . . .
3 . . . . .
. . . . .
. . . . .
6 . . 1 . . . .

. . . . .
. . . . . 5 .
. . . . .
. . . . .
3 . . . . .
. . . . .
. . . . .
. . . 6 . . . .

. . . . .
. . . . . 5 .
. . . . .
. . . . .
. . . . .
. . . . .
. . . 3 . . . .

. . . . .
. . . . . 5 .
. . . . .
. . . . .
. . . . .
. . . . .
. . . 3 . . . .
=== Menú Principal ===
1. Iniciar nuevo juego
2. Revisar juego anterior
3. Salir
Elige una opción (1-3): 2
Nombre del juego a revisar: estrella
=== Revisión de la partida 'estrella' ===
=== Estado inicial del tablero ===
. . . . .
. . . 3 . . 7 .
. . . . .
. . . . .
4 . . 1 . . 5 .
. . . . .
. . . . .
6 . . 2 . . . .
Turno de la Reina 1 (d4)
  Ataca a Reina 2 en d1
. . . . .
. . . 3 . . 7 .
. . . . .
. . . . .
4 . . . . . 5 .
. . . . .
. . . . .
6 . . 1 . . . .

```

```

Turno de la Reina 3 (d7)
  Ataca a Reina 4 en a4
. . . . .
. . . . . 7 .
. . . . .
. . . . .
3 . . . . . 5 .
. . . . .
. . . . .
6 . . 1 . . . .
Turno de la Reina 5 (g4)
  Ataca a Reina 7 en g7
. . . . .
. . . . . 5 .
. . . . .
. . . . .
3 . . . . .
. . . . .
. . . . .
6 . . 1 . . . .
Turno de la Reina 6 (a1)
  Ataca a Reina 1 en d1
. . . . .
. . . . . 5 .
. . . . .
. . . . .
3 . . . . .
. . . . .
. . . . .
. . . 6 . . . .
Turno de la Reina 3 (a4)
  Ataca a Reina 6 en d1
. . . . .
. . . . . 5 .
. . . . .
. . . . .
. . . . .
. . . . .
. . . 3 . . . .
Turno de la Reina 5 (g7)
  No puede atacar a nadie. Pasa su turno.
Turno de la Reina 3 (d1)
  No puede atacar a nadie. Pasa su turno.
Turno de la Reina 5 (g7)
  No puede atacar a nadie. Pasa su turno.
Fin del juego. Estado final:
. . . . .
. . . . . 5 .
. . . . .
. . . . .
. . . . .
. . . . .
. . . 3 . . . .
Reinas sobrevivientes:
  Reina 3 en d1
  Reina 5 en g7

=== Menú Principal ===
1. Iniciar nuevo juego
2. Revisar juego anterior
3. Salir
Elige una opción (1-3): 3
¡Hasta luego!

```